UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Arquitectura, tecnologías, pruebas y CI/CD

Guía paso a paso del sistema completo

Plataforma de Gestión Académica – Unidad Educativa "Andoas" (Puyo, Pastaza)

Asignatura: Construcción y Evolución del Software

NRC: 36847

Docente: Ing. Mayra Alvarez

Integrantes: Christian Marcelo Acuña Gamboa

Abner David Arboleda Roman

Christian Mateo Bonifaz Vásquez

Fecha: 25 de octubre de 2026

Documento complementario al Entregable 3 (Plan de construcción) y al archivo `Planificacion_v2.xlsx`.

Índice

1 Cómo leer este documento **2**

2 Visión general de la arquitectura **2**
2.1 Responsabilidad de cada microservicio 2
2.2 Reglas de diseño que se cumplen en todo el proyecto 3

3 Repositorio, ramas y forma de trabajo **4**

4 Microservicio de identidad (Java + Spring Boot) **5**

5 Microservicio académico (TypeScript + NestJS) **6**
5.1 Paquete de cálculo de notas 6

6 Microservicio de notificaciones (Python + FastAPI) **7**

7 Comunicación entre servicios **8**
7.1 Llamadas síncronas (REST) 8
7.2 Eventos asíncronos (RabbitMQ) con patrón Outbox 8

8 Base de datos: PostgreSQL **8**

9 Frontend: PWA en React + TypeScript **9**

10 Trabajo sin conexión y sincronización **9**

11 API Gateway, caché y documentos **10**
11.1 Traefik como API Gateway 10
11.2 Redis 10
11.3 Gotenberg y exceljs 10

12 Contenedores con Docker **10**

13 Seguridad aplicada **11**

14 Estrategia de pruebas **12**

15 Rendimiento y pruebas de carga **13**
15.1 Cálculo de capacidad 13
15.2 Herramienta: k6 13

16 Monitoreo (observabilidad) **15**

17 Integración y entrega continua (CI/CD) **15**

18 Ambientes **16**

19 IDEs y herramientas por integrante **16**

20 Guía de arranque local **17**

21 Cronograma actualizado **18**

22 Trazabilidad requisitos – servicio – prueba **18**

23 Correcciones pendientes en los entregables **19**

Glosario **20**

1 Cómo leer este documento

Este documento explica, paso a paso, **qué tecnologías usará el proyecto, por qué se eligió cada una, cómo se configuran, cómo se prueban y cómo llegan a producción**. Responde a los requisitos indicados por la docente:

1. Al menos **tres microservicios**, cada uno con su **base de datos relacional**.
2. Uso de **varios lenguajes de programación** y **varios IDEs**.
3. Un flujo de **integración y entrega continua (CI/CD)**.
4. Capacidad para atender a **400 estudiantes y 60 docentes al mismo tiempo**, demostrada con **pruebas de carga**.

Cada sección sigue el mismo orden: *qué es*, *por qué se usa aquí*, *pasos para implementarlo* y *cómo se prueba*. Los requisitos citados (RF, RNF) y los riesgos (R, V) son los del Entregable 3 y del Taller de vulnerabilidades.

Necesidad	Cómo se cumple
3 microservicios con BD relacional	`ms-identidad`, `ms-academico` y `ms-notificaciones`, cada uno con su base PostgreSQL.
Varios lenguajes	Java (identidad), TypeScript (académico y PWA), Python (notificaciones), SQL y YAML.
Varios IDEs	IntelliJ IDEA, Visual Studio Code, PyCharm y DBeaver.
CI/CD	GitHub Actions: un pipeline por servicio, despliegue automático a `test` y con aprobación a producción.
400 estudiantes simultáneos	Servicios sin estado, réplicas, caché, índices y pruebas de carga con k6 y Grafana.
Internet inestable	PWA con almacenamiento local cifrado y sincronización por campo.

Tabla 1: Resumen de cómo se cumplen los requisitos de la docente y del cliente.

2 Visión general de la arquitectura

La arquitectura es de **microservicios** con una **PWA** (aplicación web instalable) como cliente. Todas las peticiones entran por un **API Gateway**, que reparte el tráfico a cada servicio. Los servicios se comunican de dos formas: **REST** cuando necesitan una respuesta inmediata y **eventos por RabbitMQ** cuando algo debe ocurrir “después”, como enviar un aviso por WhatsApp.

2.1 Responsabilidad de cada microservicio

Servicio	Tecnología	Responsabilidad	Requisitos
`ms-identidad`	Java 21, Spring Boot, Spring Security, Flyway	Cuentas, roles, inicio de sesión, emisión de JWT, bloqueo tras 3 intentos, cambio de clave inicial, auditoría de accesos.	RF-01 a RF-03, RNF-04, RNF-12

Servicio	Tecnología	Responsabilidad	Requisitos
`ms-academico`	Node.js, NestJS, TypeScript, Prisma	Año lectivo, cursos, matrícula e importación de nóminas, planificación con estados, calificaciones, tareas, asistencia, comportamiento, reportes, exportación Word/PDF/Excel y sincronización sin conexión.	RF-04 a RF-28, RF-36 a RF-38, RNF-01, RNF-02, RNF-09, RNF-10
`ms-notificaciones`	Python, FastAPI, SQLAlchemy, Alembic	Contactos y consentimiento de representantes, plantillas, cola de envío, antiduplicados, historial y estado de entrega de WhatsApp.	RF-29 a RF-35, RNF-05
`web` (PWA)	React, TypeScript, Vite, Workbox, Dexie	Interfaz de los cinco roles en computadora y celular; trabajo sin conexión.	RNF-01, RNF-03, RNF-07

Tabla 2: Reparto de responsabilidades.

¿Por qué?

- **Se divide por dominio de negocio**, no por capa técnica: identidad, gestión académica y comunicación cambian por razones distintas y tienen riesgos distintos.
- **Notificaciones queda aislado** porque depende de un proveedor externo (riesgo R10): si WhatsApp falla o tarda, el registro de notas sigue funcionando.
- **Identidad queda aislado** porque concentra los datos más sensibles (contraseñas) y la lógica de seguridad (riesgo R9).
- **No se crean más de tres servicios**: con tres personas y doce semanas, cada servicio extra suma un pipeline, una base y un despliegue más sin aportar valor al colegio.

2.2 Reglas de diseño que se cumplen en todo el proyecto

1. **Una base de datos por servicio.** Ningún servicio lee ni escribe la base de otro. Si necesita un dato ajeno, lo pide por API o lo recibe por evento.
2. **Servicios sin estado.** La sesión vive en el JWT; así se pueden levantar réplicas.
3. **Identificadores UUID** generados también en el cliente, para poder crear registros sin conexión sin choques de IDs.
4. **Campo `version`** en toda tabla editable, para detectar conflictos al sincronizar.
5. **Contrato primero:** cada servicio publica su especificación OpenAPI antes de programar.
6. **Validación en el servidor** de toda entrada y todo permiso (RNF-13).

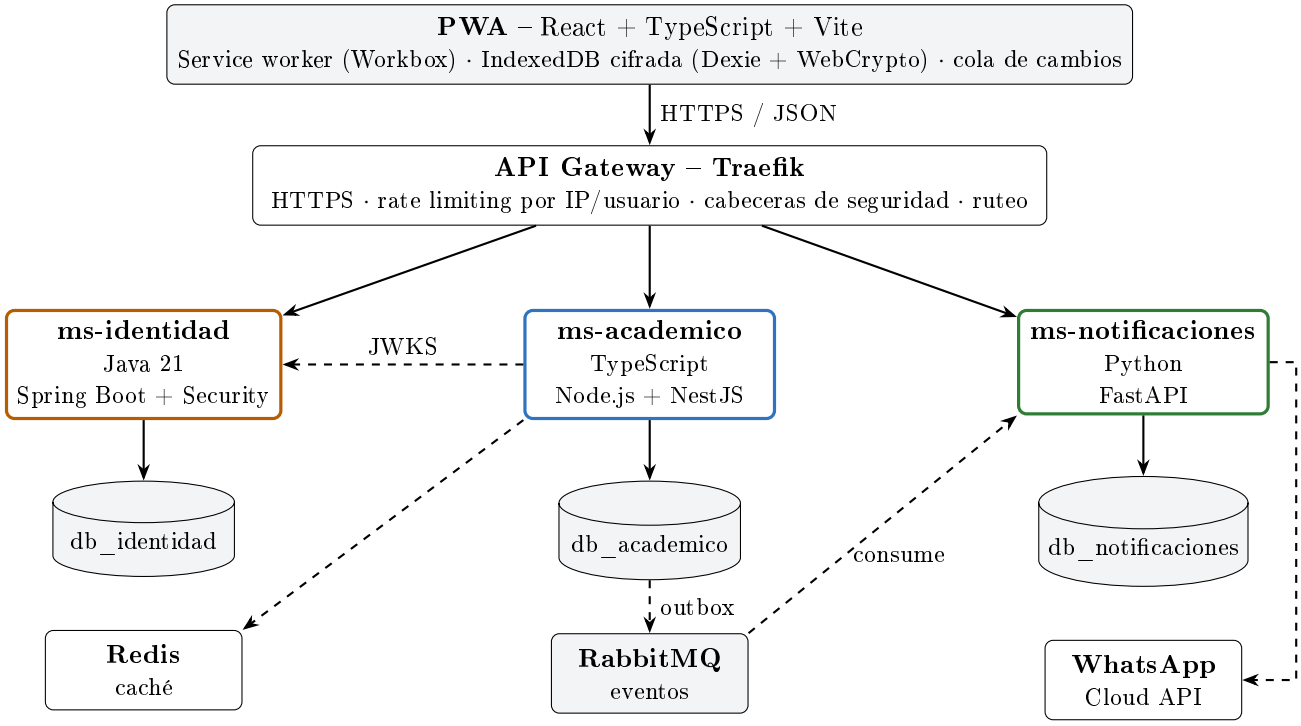


Figura 1: Diagrama de contenedores (nivel 2 del modelo C4). Líneas continuas: llamadas síncronas; líneas discontinuas: eventos, caché o validación de tokens.

3 Repositorio, ramas y forma de trabajo

Paso 1 – Estructura del monorepo

Un solo repositorio de GitHub contiene todo. Así una historia de usuario que toca la PWA y un servicio se entrega en un solo pull request.

```
plataforma-andoas/
|-- apps/web/                   # PWA (React + TypeScript)
|-- services/
|   |-- identidad/              # Java 21 + Spring Boot (Maven)
|   |-- academico/              # NestJS + Prisma
|   `-- notificaciones/         # FastAPI + SQLAlchemy
|-- packages/calculo-notas/     # motor de cálculo compartido (TS)
|-- openapi/                    # contratos de cada servicio
|-- infra/
|   |-- docker-compose.yml
|   |-- docker-compose.test.yml
|   |-- traefik/                # rutas, TLS y límites
|   |-- postgres/init.sql
|   `-- grafana/                # paneles
|-- tests/
|   |-- e2e/                    # Playwright
|   `-- carga/                  # scripts k6
|-- docs/  (arquitectura, seguridad, pruebas, sprints)
`-- .github/workflows/          # un pipeline por componente
```

Paso 2 – Ramas y reglas de protección

- `dev`: trabajo diario. Cada *push* ejecuta el CI del componente modificado.
- `test`: recibe `dev` al final de cada sprint mediante PR; requiere todos los checks en verde

y una revisión. Al unirse, se despliega automáticamente el ambiente de pruebas.

- `main`: solo versiones aceptadas por el vicerrector; requiere aprobación manual en el *environment* `produccion`. Cada unión crea un *release* (`v1.0.0`).
- Archivo `CODEOWNERS`: cada carpeta tiene un responsable que revisa los PR.
- Mensajes de commit con *Conventional Commits*: `feat(academico): calcular nota final`.

4 Microservicio de identidad (Java + Spring Boot)

Qué es. Spring Boot es el framework más usado en Java para servicios web; Spring Security resuelve autenticación y autorización de forma probada. Se usa **Java 21 LTS** con la versión estable vigente de **Spring Boot 4**. Las migraciones de la base se hacen con **Flyway**.

Pasos de implementación

1. Crear el proyecto en `https://start.spring.io` con Maven y las dependencias *Spring Web, Spring Security, OAuth2 Resource Server, Spring Data JPA, Validation, PostgreSQL Driver, Flyway, Actuator* y *Testcontainers*.
2. Definir las tablas en `src/main/resources/db/migration/V1__init.sql`: `usuario`, `rol`, `usuario_rol`, `intento_login` y `auditoria_acceso`.
3. Guardar contraseñas con **Argon2** o **BCrypt** (`PasswordEncoder`).
4. Implementar `POST /auth/login`: verifica la clave, cuenta intentos fallidos y bloquea la cuenta 15 minutos al tercer fallo (RNF-12).
5. Emitir un **JWT firmado con RS256** de vida corta (15 min) y un *refresh token* guardado en cookie `HttpOnly`. El token incluye `sub`, `roles` y `debeCambiarClave`.
6. Publicar la clave pública en `GET /.well-known/jwks.json`. Los otros servicios validan los tokens con esa clave, sin llamar a identidad en cada petición.
7. Generar el nombre de usuario a partir de nombre y apellido, con número si ya existe (RF-02).
8. Exponer `/actuator/health` y `/actuator/prometheus` para monitoreo.

Atención

La cédula como clave inicial (RF-03) es un riesgo identificado (R9). Se mitiga obligando a cambiarla en el primer ingreso: mientras `debeCambiarClave` sea verdadero, el token solo permite el endpoint de cambio de clave.

Cómo se prueba: JUnit 5 para la lógica, **Testcontainers** para levantar un PostgreSQL real en las pruebas de integración, **JaCoCo** para la cobertura y **OWASP Dependency-Check** o Trivy para las dependencias.

```
@Test
void bloqueaLaCuentaTrasTresIntentosFallidos() {
    for (int i = 0; i < 3; i++) {
        assertThrows(CredencialesInvalidas.class,
            () -> authService.login("mperez", "claveIncorrecta"));
    }
    assertThrows(CuentaBloqueada.class,
        () -> authService.login("mperez", "claveCorrecta"));
}
```

5 Microservicio académico (TypeScript + NestJS)

Qué es. NestJS es un framework para Node.js que organiza el código en módulos, controladores y servicios (muy parecido a Spring). Se usa con **Node.js 24 LTS**, **TypeScript** y el ORM **Prisma** para la base de datos.

¿Por qué?

El motor de cálculo de notas debe dar **exactamente** el mismo resultado en el servidor y en la PWA cuando el docente trabaja sin conexión. Si ambos están en TypeScript, se usa **el mismo paquete** (`packages/calculo-notas`) en los dos lados y se prueba una sola vez contra la hoja oficial.

Pasos de implementación

1. Crear el proyecto: `npx @nestjs/cli new academico` y agregar Prisma (`npm i prisma @prisma/client`).
2. Organizar módulos: `anio-lectivo`, `cursos`, `matricula`, `planificacion`, `calificaciones`, `tareas`, `asistencia`, `comportamiento`, `reportes`, `sync` y `outbox`.
3. Definir el esquema en `prisma/schema.prisma` y crear migraciones con `npx prisma migrate dev`.
4. Validar tokens con `passport-jwt` y `jwks-rsa`, y crear guardas por rol (`@Roles('DOCENTE')`) más una verificación de propiedad: un docente solo califica sus materias.
5. Validar todas las entradas con `class-validator` (DTOs) y documentar con `@nestjs/swagger`.
6. Planificación: implementar la máquina de estados *Borrador* → *En revisión* → *(Con observaciones* → *Borrador)* → *Aprobado*. Una transición no permitida devuelve 409.
7. Exportación: Word con la librería `docx` sobre la plantilla del Ministerio; PDF convirtiendo ese Word con **Gotenberg** (contenedor con LibreOffice), así el PDF y el Word tienen el mismo formato; Excel con `exceljs`.
8. Auditoría: cada cambio de nota o planificación aprobada inserta una fila en `auditoria` con usuario, fecha, valor anterior y nuevo (RNF-09, V4).

5.1 Paquete de cálculo de notas

Los números decimales de JavaScript tienen errores de precisión ($0{,}1+0{,}2 \neq 0{,}3$). Por eso el cálculo se hace con **decimal.js** y la regla de redondeo es un parámetro configurable (RNF-10).

```
import Decimal from 'decimal.js';

export type Regla = 'TRUNCAR' | 'REDONDEAR';
export interface Ponderacion { aportes: number; proyecto: number; examen: number; regla: Regla; }

export function notaFinal(aportes: number, proyecto: number, examen: number, p: Ponderacion):
    number {
  const bruto = new Decimal(aportes).mul(p.aportes)
    .plus(new Decimal(proyecto).mul(p.proyecto))
    .plus(new Decimal(examen).mul(p.examen));
  const modo = p.regla === 'TRUNCAR' ? Decimal.ROUND_DOWN : Decimal.ROUND_HALF_UP;
  return bruto.toDecimalPlaces(2, modo).toNumber();
}
```

Atención

El Entregable 1 dice “redondear” y RF-18 dice “truncar”. Antes del 17/11 deben confirmar con la hoja oficial del colegio cuál regla se usa y dejarla escrita en un solo lugar.

Prueba parametrizada contra la hoja oficial (indicador I-05):

```
import casos from './fixtures/hoja-oficial-9noA.json'; // datos anonimizados
describe.each(casos)('estudiante $id', (c) => {
  it('coincide con la hoja oficial', () => {
    expect(notaFinal(c.aportes, c.proyecto, c.examen, PONDERACION_COLEGIO)).toBe(c.esperado);
  });
});
```

6 Microservicio de notificaciones (Python + FastAPI)

Qué es. FastAPI es un framework moderno de Python para APIs; genera la documentación OpenAPI automáticamente y valida datos con Pydantic. Se usa con **SQLAlchemy** (ORM), **Alembic** (migraciones) y **aio-pika** para consumir eventos de RabbitMQ.

Pasos de implementación

1. Crear el entorno: `python -m venv .venv` e instalar `fastapi uvicorn sqlalchemy alembic psycopg aio-pika httpx pydantic-settings`.
2. Tablas: `contacto` (número, representante, verificado), `consentimiento` (fecha, medio, estado), `plantilla`, `notificacion` (motivo, estado de entrega, intentos) y `clave_idempotencia`.
3. **Adaptador de proveedor**: una interfaz `ProveedorMensajeria` con dos implementaciones, `WhatsAppCloud` (real) y `ProveedorSimulado` (pruebas). Así se cumple el plan B del riesgo R10.
4. Consumidor de eventos: escucha `nota.registrada` y `falta.registrada`. Antes de enviar verifica: número verificado, consentimiento vigente y que no exista otro aviso con la misma clave `(estudiante, materia, motivo, día)` (RF-34).
5. Plantillas **sin detalles de la nota** (política de seguridad, V3): “Su representado tiene un aviso académico en Matemática. Revise el portal.”
6. Webhook `POST /webhooks/whatsapp` para actualizar el estado de entrega (RF-33).
7. Reintentos con espera creciente y una **cola de mensajes fallidos** (DLQ) para revisar a mano lo que no se pudo entregar.

Cómo se prueba: pytest con `httpx.AsyncClient`, Testcontainers para PostgreSQL y RabbitMQ, el proveedor simulado y `pip-audit` para dependencias.

```
async def test_no_envia_si_no_hay_consentimiento(cliente, proveedor_simulado):
    await crear_contacto(numero="+593990000000", consentimiento=False)
    await publicar_evento("nota.registrada", estudiante_id=EST, promedio=6.5)
    await procesar_cola()
    assert proveedor_simulado.enviados == []
```

7 Comunicación entre servicios

7.1 Llamadas síncronas (REST)

La PWA llama a los servicios a través del gateway. Entre servicios, solo hay una llamada síncrona importante: `ms-academico` crea la cuenta de un docente o estudiante en `ms-identidad` usando un **token de servicio** (flujo *client credentials*), nunca con el token del usuario.

7.2 Eventos asíncronos (RabbitMQ) con patrón Outbox

¿Por qué?

Si `ms-academico` guardara la nota y luego intentara publicar el evento, una caída entre los dos pasos dejaría una nota sin aviso. Con el **patrón Transactional Outbox** el evento se guarda en la **misma transacción** que la nota, en la tabla `outbox`; un proceso aparte la lee y publica en RabbitMQ. Esto también resuelve RF-35: lo que se registró sin conexión genera sus eventos cuando llega al servidor.

```
# Exchange "academico.eventos" (tipo topic)
#   nota.registrada   → cola "notificaciones.notas"
#   falta.registrada  → cola "notificaciones.faltas"
#   cola de fallidos  → "notificaciones.dlq"
# Ejemplo de mensaje (JSON):
{ "id": "0193f0c2-...", "tipo": "nota.registrada", "ocurrido": "2026-11-18T10:21:00-05:00",
  "estudianteId": "0193...", "materiaId": "0193...", "promedio": 6.85 }
```

8 Base de datos: PostgreSQL

Qué es. PostgreSQL es un motor relacional libre, robusto y con muy buen rendimiento. Se usa **una instancia con tres bases y tres usuarios** para no pagar tres servidores, manteniendo el aislamiento.

Pasos

1. Script `infra/postgres/init.sql`:

```
CREATE USER identidad_app WITH PASSWORD :'pass_identidad';
CREATE DATABASE db_identidad OWNER identidad_app;
CREATE USER academico_app WITH PASSWORD :'pass_academico';
CREATE DATABASE db_academico OWNER academico_app;
CREATE USER notif_app WITH PASSWORD :'pass_notif';
CREATE DATABASE db_notificaciones OWNER notif_app;
REVOKE CONNECT ON DATABASE db_identidad FROM PUBLIC;  -- repetir para cada base
```

2. Convenciones en todas las tablas editables: `id UUID`, `version INTEGER`, `creado_en`, `actualizado_en`, `actualizado_por`.
3. Índices en las columnas de búsqueda frecuente: `calificacion(estudiante_id, materia_id, trimestre)`, `asistencia(curso_id, fecha)`, `matricula(curso_id, anio_lectivo_id)`.
4. Reportes pesados (valoración de logros, tablero) con **vistas SQL** y caché en Redis.
5. Revisar consultas lentas con `EXPLAIN ANALYZE` y la extensión `pg_stat_statements`.
6. Respaldos diarios con `pg_dump` de cada base y una prueba de restauración documentada (RNF-08, R12).

Servicio	Migraciones	Comando
Identidad	Flyway	Se aplican solas al iniciar Spring Boot.
Académico	Prisma Migrate	`npx prisma migrate deploy`
Notificaciones	Alembic	`alembic upgrade head`

Tabla 3: Herramienta de migración por servicio.

9 Frontend: PWA en React + TypeScript

Qué es. React es la librería de interfaces más usada; Vite compila rápido; una PWA se puede instalar en el celular como si fuera una app y funciona sin conexión gracias al *service worker*.

Pasos de implementación

1. Crear el proyecto: `npm create vite@latest web - -template react-ts`.
2. Librerías: **React Router** (rutas por rol), **TanStack Query** (datos del servidor con caché), **React Hook Form** + **Zod** (formularios y validación), **Tailwind CSS** (estilos), **vite-plugin-pwa** (service worker con Workbox) y **Dexie** (IndexedDB).
3. Reutilizar las pantallas validadas del prototipo (`standalone_demo v2.html`) como referencia visual.
4. Diseño primero para celular (supuesto de docentes con celular) y acciones frecuentes en máximo tres pasos (RNF-03).
5. Ocultar botones según el rol es solo comodidad: el permiso real siempre lo valida el servidor.

10 Trabajo sin conexión y sincronización

Es el requisito de mayor riesgo (Wiegers: costo 9, riesgo 9). Por eso, aunque se construye en el sprint 9, **su diseño empieza en el sprint 3**: UUID, `version` y endpoints `/sync` definidos en el contrato.

Funcionamiento paso a paso

1. **Caché de la aplicación:** Workbox guarda HTML, JS, CSS e íconos. La PWA abre sin internet.
2. **Datos locales:** Dexie guarda en IndexedDB los cursos, estudiantes y notas del docente. Los datos se cifran con **WebCrypto (AES-GCM)** y se borran al cerrar sesión (V1, R4).
3. **Cola de cambios:** cada edición sin conexión se guarda como `{id, entidad, baseVersion, campos, fechaCliente}`.
4. **Push:** al volver la conexión, la PWA envía la cola a `POST /sync/push`. Cada cambio lleva su UUID, así reenviarlo no lo duplica (idempotencia).
5. **Detección de conflicto:** si `baseVersion` coincide con la versión del servidor, se aplica. Si no, se compara **campo por campo**: los campos que nadie más tocó se aplican; si el mismo campo cambió en ambos lados, gana el registro más reciente y el valor perdedor se guarda en la tabla `conflicto_sync` y se muestra al docente (V2, R5).
6. **Pull:** la PWA pide `GET /sync/pull?desde=<version>` y recibe solo lo que cambió.

7. **Eventos:** los cambios sincronizados generan sus eventos en el outbox, y los avisos de WhatsApp salen en ese momento (RF-35).

Atención

El cifrado en el navegador protege ante pérdida del equipo o un computador compartido, pero no ante alguien que usa la sesión abierta. Por eso también hay cierre de sesión por inactividad.

11 API Gateway, caché y documentos

11.1 Traefik como API Gateway

- Ruteo por prefijo: `/api/identidad`, `/api/academico`, `/api/notificaciones` y `/` para la PWA.
- **HTTPS** con certificados de Let’s Encrypt en producción (RNF-14).
- **Rate limiting** (V5, R8): por ejemplo, promedio 50 peticiones/s por IP con ráfaga de 100, y un límite más estricto en `/auth/login` (5 por minuto por IP).
- Cabeceras de seguridad: HSTS, `X-Content-Type-Options`, `Content-Security-Policy`.
- Balanceo entre réplicas de `ms-academico`.

11.2 Redis

Guarda por 60 segundos el resultado del tablero del administrador y de la valoración de logros. Cuando se registra una nota, se borra la entrada de ese curso. Es opcional: si las pruebas de carga muestran buenos tiempos sin Redis, se puede omitir.

11.3 Gotenberg y exceljs

Gotenberg es un contenedor que convierte Word a PDF con LibreOffice; así el PDF conserva el formato del Ministerio. Excel se genera con `exceljs`, protegiendo las celdas contra inyección de fórmulas (valores que empiezan con `=`, `+`, `-` o `@`).

12 Contenedores con Docker

Cada componente tiene un `Dockerfile` **multietapa** (una etapa compila, otra ejecuta con una imagen pequeña y un usuario sin privilegios). Todo se levanta con Docker Compose.

```
services:
  traefik:
    image: traefik:v3
    ports: ["80:80", "443:443"]
    volumes: ["./traefik:/etc/traefik", "/var/run/docker.sock:/var/run/docker.sock:ro"]
  postgres:
    image: postgres:17
    environment: { POSTGRES_PASSWORD: ${PG_ROOT_PASSWORD} }
    volumes: ["pgdata:/var/lib/postgresql/data", "./postgres/init.sql:/docker-entrypoint-initdb.d/
    init.sql"]
    healthcheck: { test: ["CMD", "pg_isready"], interval: 5s }
  rabbitmq:
    image: rabbitmq:4-management
  redis:
    image: redis:7-alpine
  gotenberg:
    image: gotenberg/gotenberg:8
  identidad:
```

```
    image: ghcr.io/andoas/identidad:${TAG}
    environment: { DB_URL: jdbc:postgresql://postgres:5432/db_identidad }
    depends_on: { postgres: { condition: service_healthy } }
    labels: ["traefik.http.routers.identidad.rule=PathPrefix(‘/api/identidad‘)"]
  academico:
    image: ghcr.io/andoas/academico:${TAG}
    deploy: { replicas: 2 }
    labels: ["traefik.http.routers.academico.rule=PathPrefix(‘/api/academico‘)"]
  notificaciones:
    image: ghcr.io/andoas/notificaciones:${TAG}
    labels: ["traefik.http.routers.notif.rule=PathPrefix(‘/api/notificaciones‘)"]
  web:
    image: ghcr.io/andoas/web:${TAG}
    labels: ["traefik.http.routers.web.rule=PathPrefix(‘/‘)"]
volumes: { pgdata: {} }
```

Las contraseñas nunca van en el repositorio: se leen de un archivo `.env` local (ignorado por Git) o de los *secrets* de GitHub en el pipeline.

13 Seguridad aplicada

Riesgo	Control técnico	Cómo se verifica
V1 / R4 Datos sin cifrar en el equipo	WebCrypto AES-GCM en IndexedDB, borrado al cerrar sesión, cierre por inactividad.	Revisión de IndexedDB y prueba e2e de cierre de sesión.
V2 / R5 Conflictos al sincronizar	Versiones, resolución por campo y tabla `conflicto_sync`.	Pruebas de dos docentes editando lo mismo.
V3 / R6 Aviso a persona equivocada	Número verificado, consentimiento registrado, plantillas sin datos de notas.	pytest: no se envía sin consentimiento.
V4 / R7 Manipulación de notas	Tabla de auditoría y bloqueo al cerrar el trimestre.	Consulta de auditoría y prueba de edición bloqueada.
V5 / R8 Abuso de peticiones	Rate limiting en Traefik por IP y usuario; límite estricto en login.	Prueba k6 que supera el límite y espera 429.
R9 Robo de cuentas	Argon2/BCrypt, bloqueo tras 3 intentos, cambio obligatorio de la clave inicial, JWT de vida corta.	Pruebas JUnit del login.
RNF-13 Validación	DTOs con validación en los tres servicios; consultas parametrizadas por el ORM.	SonarQube y OWASP ZAP.
RNF-15 Desarrollo seguro	Gitleaks, análisis de dependencias, Trivy y quality gate obligatorios.	Reglas de protección de ramas.

Tabla 4: Relación entre vulnerabilidades, controles y verificación.

Se agrega **Dependabot** para que las dependencias de Java, npm, Python y las imágenes Docker se actualicen con PRs automáticos.

14 Estrategia de pruebas

Se sigue la **pirámide de pruebas**: muchas pruebas unitarias rápidas, menos pruebas de integración y pocas pruebas de punta a punta, más pruebas de seguridad y de carga.

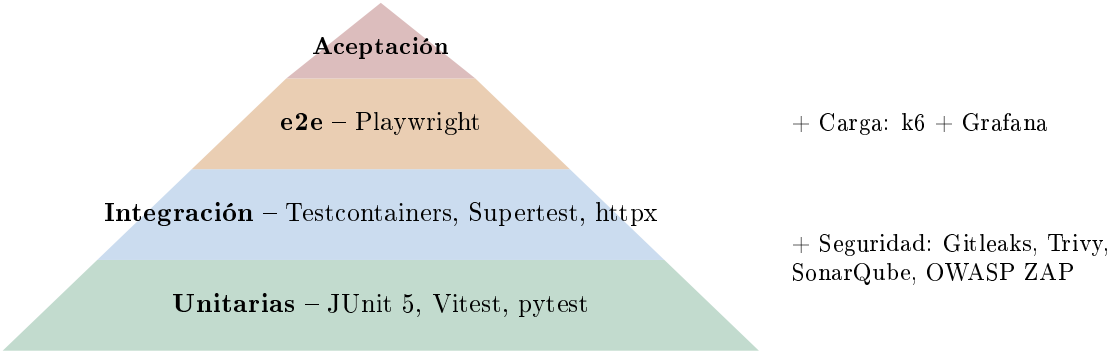


Figura 2: Pirámide de pruebas del proyecto.

Nivel	Identidad (Java)	Académico (TS)	Notificaciones (Py)	PWA
Unitarias	JUnit 5 + Mockito	Vitest	pytest	Vitest + Testing Library
Integración	Spring Boot Test + Testcontainers	Supertest + Testcontainers	pytest + Testcontainers	–
Contrato	Validación de cada servicio contra su OpenAPI publicado (por ejemplo, con Schemathesis)			–
e2e	Playwright contra el ambiente `test` (Chrome, Edge, Firefox y vista de celular)			
Cobertura	JaCoCo	Vitest (v8)	coverage.py	Vitest (v8)
Dependencias	Dependency-Check / Trivy	npm audit	pip-audit	npm audit
Seguridad	Gitleaks (secretos), SonarQube (SAST), Trivy (imágenes), OWASP ZAP (DAST)			
Carga	k6 contra el ambiente `test`, con métricas en Grafana			

Tabla 5: Herramienta de prueba por nivel y por componente.

Qué se prueba siempre

1. **Exactitud** (RNF-02): todos los casos de la hoja oficial anonimizada coinciden al 100 %.
2. **Autorización** (RNF-04): cada endpoint se prueba con un rol sin permiso (403) y con un recurso ajeno (por ejemplo, un estudiante pidiendo las notas de otro).
3. **Estados** de la planificación: toda transición inválida devuelve 409.
4. **Consentimiento** (RNF-05): no se envía a números sin autorización.
5. **Sin conexión** (RNF-01): Playwright desconecta la red (`context.setOffline(true)`), registra notas, reconecta y verifica que llegaron al servidor.
6. **Compatibilidad** (RNF-07): Playwright ejecuta las pruebas e2e en Chromium, Firefox

y WebKit, más una prueba manual en celular en modo avión.

Ejemplo e2e del modo sin conexión (Playwright):

```
test('registra notas sin internet y sincroniza', async ({ page, context }) => {
  await iniciarSesion(page, 'docente.prueba');
  await page.goto('/libreta/9noA/matematica');
  await context.setOffline(true);
  await page.getByLabel('Deberes - Vega Ruiz').fill('9');
  await expect(page.getByText('Sin conexión: 1 cambio pendiente')).toBeVisible();
  await context.setOffline(false);
  await expect(page.getByText('Sincronizado')).toBeVisible();
  const nota = await api.obtenerNota('Vega Ruiz', 'deberes');
  expect(nota).toBe(9);
});
```

15 Rendimiento y pruebas de carga

15.1 Cálculo de capacidad

- Usuarios simultáneos: $400 + 60 = 460$.
- Si cada usuario hace en promedio una petición cada 10 segundos: $460/10 \approx 46$ peticiones por segundo.
- En un pico (todos revisan sus notas al publicarse), se multiplica por 4: ≈ 185 peticiones por segundo.
- **Meta de diseño: 200 peticiones/s con p95 menor a 500 ms y menos del 1 % de errores.**

Esto es alcanzable con un servidor de 4 vCPU y 8 GB de RAM si se aplica lo siguiente: servicios sin estado con réplicas, *pool* de conexiones (HikariCP en Java, el pool de Prisma, o PgBouncer), índices en las consultas frecuentes, consultas paginadas y caché para los tableros.

15.2 Herramienta: k6

k6 es una herramienta libre para pruebas de carga. Los escenarios se escriben en JavaScript, se versionan en `tests/carga/` y sus métricas se envían a Grafana.

```
// tests/carga/carga-460.js   →   k6 run -e BASE_URL=https://test.andoas.ec carga-460.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  scenarios: {
    estudiantes: { executor: 'ramping-vus', exec: 'estudiante', startVUs: 0,
      stages: [ { duration: '3m', target: 400 }, { duration: '10m', target: 400 },
                { duration: '2m', target: 0 } ] },
    docentes: { executor: 'constant-vus', exec: 'docente', vus: 60, duration: '15m' },
  },
  thresholds: {
    http_req_duration: ['p(95)<500'],   // 95 % de respuestas en menos de 500 ms
    http_req_failed:   ['rate<0.01'],   // menos de 1 % de errores
  },
};

const BASE = __ENV.BASE_URL;
function login(usuario) {
  const r = http.post(`${BASE}/api/identidad/auth/login`,
    JSON.stringify({ usuario, clave: __ENV.CLAVE_PRUEBA }),
```

Escenario	Qué simula	Criterio de éxito
Humo	5 usuarios durante 1 minuto. Verifica que el script funciona.	0 errores
Línea base	100 usuarios (sprint 5). Sirve para comparar sprint a sprint.	p95 $<$ 500 ms
Carga	Sube a 400 estudiantes y 60 docentes en 3 min y se mantiene 10 min.	p95 $<$ 500 ms, errores $<$ 1 %
Pico	Los 400 estudiantes inician sesión casi al mismo tiempo.	Sin caídas; p95 $<$ 1 s
Estrés	Aumenta usuarios hasta que el sistema falla.	Documentar el punto de quiebre
Resistencia	460 usuarios durante 1 hora.	Sin aumento de memoria ni errores

Tabla 6: Escenarios de prueba de carga.

```
    { headers: { 'Content-Type': 'application/json' } });
  return { headers: { Authorization: `Bearer ${r.json('accessToken')}` } };
}

export function estudiante() {
  const auth = login(`est${String(__VU).padStart(3, '0')}`);   // usuarios del seed
  const r = http.get(`${BASE}/api/academico/estudiante/tablero`, auth);
  check(r, { 'tablero 200': (x) => x.status === 200 });
  sleep(10);                                                    // tiempo de lectura
  http.get(`${BASE}/api/academico/estudiante/calendario`, auth);
  sleep(10);
}

export function docente() {
  const auth = login(`doc${String(__VU % 60).padStart(2, '0')}`);
  http.get(`${BASE}/api/academico/libreta?curso=9noA&materia=MAT`, auth);
  sleep(5);
  http.put(`${BASE}/api/academico/calificaciones/aporte`,
    JSON.stringify({ /* aporte de prueba */ }), auth);
  sleep(15);
}
```

Cómo ejecutar e interpretar

1. Cargar datos de prueba: 400 estudiantes, 60 docentes y 13 cursos (*seed*).
2. Ejecutar contra el ambiente `test`, nunca contra producción ni en cada PR. Se programa en GitHub Actions de forma manual (`workflow_dispatch`) o semanal.
3. Mirar en Grafana: tiempo de respuesta p95, peticiones/s, errores, CPU y memoria por servicio y conexiones activas de PostgreSQL.
4. Si un umbral falla, k6 termina con error y el pipeline se marca en rojo.
5. En el informe del sprint 9 se documenta el **punto de quiebre**: cuántos usuarios soporta el sistema antes de superar 1 s o 1 % de errores, y qué cuello de botella se encontró.

16 Monitoreo (observabilidad)

- **Prometheus** recoge métricas de los tres servicios: `/actuator/prometheus` (Spring), `prom-client` (NestJS) y un instrumentador de Prometheus para FastAPI.
- **Grafana** muestra paneles de tiempos de respuesta, errores, peticiones por segundo, estado de las colas de RabbitMQ y uso de la base de datos.
- **Logs en JSON** con un `traceId` que viaja en la cabecera entre servicios, para seguir una petición de punta a punta. Opcionalmente se centralizan con Loki.
- **Health checks** en cada servicio, usados por Docker para reiniciar contenedores caídos.

17 Integración y entrega continua (CI/CD)

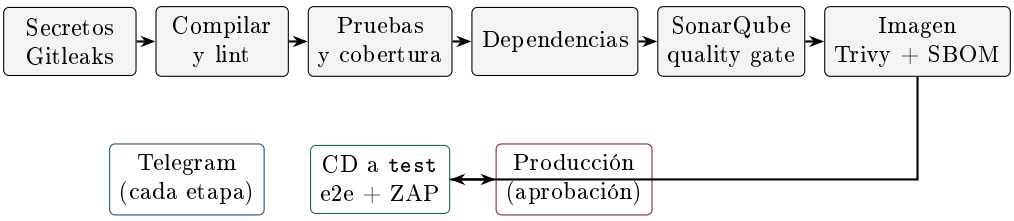


Figura 3: Etapas del pipeline de cada componente.

Pasos

1. Un workflow por componente: `ci-identidad.yml`, `ci-academico.yml`, `ci-notificaciones.yml` y `ci-web.yml`. Con `paths:` solo se ejecuta el del componente que cambió.
2. Etapas comunes: Gitleaks → compilación → pruebas con cobertura → análisis de dependencias → SonarQube → imagen Docker con Trivy y SBOM → publicación en GitHub Container Registry (GHCR).
3. **SonarQube**: un proyecto por servicio en SonarQube Cloud (gratuito para repositorios públicos) o un servidor propio; los tres lenguajes se analizan.
4. **CD a `test`**: al unir a `test`, un workflow entra por SSH al servidor de pruebas y ejecuta `docker compose pull && docker compose up -d`; luego corre Playwright y OWASP ZAP contra ese ambiente.
5. **CD a producción**: al unir a `main`, el job usa el *environment* `produccion`, que exige aprobación manual de un integrante; despliega la misma imagen ya probada (mismo *tag*) y publica el release.
6. **Telegram**: un solo mensaje por ejecución que se edita con el estado de cada etapa, sin datos sensibles.

```yaml
# .github/workflows/ci-academico.yml
name: ci-academico
on:
  push:   { branches: [dev], paths: ["services/academico/**", "packages/calculo-notas/**"] }
  pull_request: { branches: [test, main], paths: ["services/academico/**", "packages/**"] }
jobs:
  build-test:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: services/academico } }
    steps:
```

```
    - uses: actions/checkout@v4
      with: { fetch-depth: 0 }
    - uses: gitleaks/gitleaks-action@v2
      env: { GITHUB_TOKEN: "${{ secrets.GITHUB_TOKEN }}" }
    - uses: actions/setup-node@v4
      with: { node-version: 24, cache: npm }
    - run: npm ci
    - run: npm run lint && npm run build
    - run: npm test -- --coverage        # Vitest + Testcontainers
    - run: npm audit --audit-level=high
    - uses: SonarSource/sonarqube-scan-action@v5
      env: { SONAR_TOKEN: "${{ secrets.SONAR_TOKEN }}" }
    - name: Imagen y escaneo
      run: |
        docker build -t ghcr.io/andoas/academico:${{ github.sha }} .
        docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy \
          image --exit-code 1 --severity HIGH,CRITICAL ghcr.io/andoas/academico:${{ github.sha
}}
    - name: Aviso en Telegram
      if: always()
      run: ../../infra/scripts/telegram.sh "${{ job.status }}" "academico" "${{ github.ref_name
}}"
      env: { TG_TOKEN: "${{ secrets.TG_TOKEN }}", TG_CHAT: "${{ secrets.TG_CHAT }}" }
```

18 Ambientes

Ambiente	Dónde	Uso
Local	Computador de cada integrante con Docker Desktop	Desarrollo y pruebas rápidas con `docker compose up`.
Pruebas (`test`)	Servidor en la nube, 2 vCPU y 4 GB	Demostraciones al vicerrector, e2e, ZAP y pruebas de carga.
Producción (`main`)	Servidor en la nube, 4 vCPU y 8 GB	Uso real del colegio, con respaldos diarios.

Tabla 7: Ambientes del proyecto.

El sistema se despliega en la nube y no en el colegio: la conexión inestable del colegio la cubre la PWA, no el servidor. Para el servidor pueden usar los créditos del GitHub Student Developer Pack (si siguen disponibles para su cuenta) o un servidor facilitado por la universidad; el supuesto del Entregable 3 ya contempla entregar imágenes de contenedor listas para instalar.

19 IDEs y herramientas por integrante

Integrante	Foco principal	IDE y herramientas
Marcelo Acuña	Scrum Master; `ms-academico`, `ms-identidad`, `ms-notificaciones`, base de datos y pruebas de carga	**IntelliJ IDEA** (Ultimate gratis con correo universitario) para Java, **Visual Studio Code** (ESLint, Prettier, Prisma, Vitest) y **PyCharm** para Python; k6 y Grafana.

Integrante	Foco principal	IDE y herramientas
Christian Bonifaz	PWA, trabajo sin conexión, DevSecOps y CI/CD, pruebas e2e y de seguridad	**Visual Studio Code** o **WebStorm** (Playwright, Tailwind, ESLint, GitHub Actions, Docker).
Abner Arboleda	Gestión del Product Backlog, criterios de aceptación, seguimiento de sprints e indicadores	**GitHub Projects**, Visual Studio Code para la documentación en Markdown del repositorio y la hoja de indicadores.
Todos	Base de datos y APIs	**DBeaver** (PostgreSQL), **Bruno** o Postman (colecciones de API versionadas), Docker Desktop, Git y GitHub Desktop opcional.

Tabla 8: IDEs y herramientas.

Las subactividades rotan entre módulos, como indica el Entregable 3: cada integrante tiene un área principal, pero en varios días trabaja en el servicio de otro (por ejemplo, Christian escribe pruebas pytest del antiduplicados y pruebas de autorización de los endpoints que construye Marcelo).

Para que el código se vea igual sin importar el IDE, el repositorio incluye `.editorconfig`, Prettier y ESLint (TypeScript), Spotless (Java) y Ruff (Python), y el CI falla si el formato no se respeta.

20 Guía de arranque local

Primer día con el proyecto

1. Instalar: Git, Docker Desktop, JDK 21, Node.js 24 LTS, Python 3.12 o superior, k6 y el IDE.
2. Clonar: `git clone https://github.com/<organizacion>/plataforma-andoas && cd plataforma-andoas`
3. Copiar variables: `cp infra/.env.example infra/.env` y completar contraseñas locales.
4. Levantar dependencias: `docker compose -f infra/docker-compose.yml up -d postgres rabbitmq redis gotenberg traefik`
5. Identidad: `cd services/identidad && ./mvnw spring-boot:run`
6. Académico: `cd services/academico && npm ci && npx prisma migrate dev && npm run start:dev`
7. Notificaciones: `cd services/notificaciones && python -m venv .venv && pip install -r requirements.txt && alembic upgrade head && uvicorn app.main:app -reload`
8. PWA: `cd apps/web && npm ci && npm run dev`
9. Datos de prueba: `npm run seed` en académico.
10. Abrir `https://localhost` y entrar con un usuario del *seed*.

21 Cronograma actualizado

El detalle día por día (actividad, subactividad de cada integrante, entregable y evidencia) está en `Planificacion_v2.xlsx`. Se mantienen las fechas de los sprints del Entregable 3; cambia el contenido desde el 26/10 para incorporar los microservicios.

Sprint	Fechas	Hito verificable	Tecnologías
3	21/10 – 30/10	Arquitectura C4 con 3 servicios; esqueletos en Java, TypeScript y Python con sus bases; CI por servicio con Telegram; esqueleto unido a `test`.	Spring Boot, NestJS, FastAPI, React, PostgreSQL, RabbitMQ, Traefik
4	04/11 – 13/11	Imágenes escaneadas en GHCR; login con JWT validado por los tres servicios; gestión académica y matrícula (RF-01 a RF-07, RF-29).	Spring Security, JWKS, Trivy, Playwright
5	16/11 – 27/11	CD a `test`; notas idénticas a la hoja oficial; tareas, asistencia, comportamiento, parámetros y auditoría; prueba de carga de línea base.	Prisma, Vitest, Testcontainers, k6
6	30/11 – 08/12	Estándares, valoración de logros, tableros, Excel y vista del representante; caché y monitoreo.	Redis, Prometheus, Grafana
7	09/12 – 18/12	Planificación con estados, observaciones por campo, historial, Word/PDF y calendario.	docx, Gotenberg
8	21/12 – 29/12	Avisos reales por WhatsApp desde eventos (outbox), con consentimiento, antiduplicados e historial.	FastAPI, RabbitMQ, WhatsApp Cloud API
9	30/12 – 08/01	Trabajo sin conexión con cifrado y sincronización por campo; PWA instalable; prueba de carga con 460 usuarios y punto de quiebre.	Workbox, Dexie, WebCrypto, k6
10	11/01 – 15/01	Aceptación, informe de seguridad final, respaldos probados y versión v1.0.0 en producción.	GitHub Environments, ZAP

Tabla 9: Resumen del cronograma por sprint.

Atención

El alcance aumentó con los microservicios (riesgo R1). Si el tiempo no alcanza, se recortan primero los requisitos de prioridad Media: RF-08 (días no dictados), RF-26/27 (Excel de notas), RF-32 (avisos de tareas) y RF-36 (mensajería interna). Redis también es opcional si las pruebas de carga lo permiten.

22 Trazabilidad requisitos – servicio – prueba

Requisito	Servicio	Prueba que lo demuestra
RF-01 a RF-03, RNF-12	identidad	JUnit: roles, generación de usuario, cambio de clave y bloqueo.
RF-04 a RF-08	académico	Supertest: CRUD y validaciones; importación con archivo inválido.
RF-09 a RF-16	académico	Pruebas de estados (409), e2e de revisión y comparación del Word/PDF con la plantilla.
RF-17, RF-18, RNF-02	académico + paquete	Vitest parametrizado con la hoja oficial (100 %).
RF-19 a RF-22	académico	Supertest y e2e de tareas, asistencia y comportamiento.
RF-23 a RF-28	académico	Comparación con el reporte oficial; k6 del tablero.
RF-29 a RF-35, RNF-05	notificaciones	pytest con proveedor simulado; prueba real con número de prueba.
RF-37, RF-38, RNF-04	académico	Pruebas 403 de acceso a datos ajenos.
RNF-01	PWA + académico	Playwright sin conexión y prueba en celular en modo avión.
RNF-06	todos	k6: carga, pico, estrés y resistencia.
RNF-07	PWA	Playwright en Chromium, Firefox y WebKit.
RNF-08	infraestructura	Prueba de restauración de `pg_dump`.
RNF-09	todos	Consulta de las tablas de auditoría.
RNF-13 a RNF-15	todos	SonarQube, ZAP, Gitleaks, Trivy y protección de ramas.

Tabla 10: Trazabilidad.

23 Correcciones pendientes en los entregables

Al revisar los cuatro documentos y el Excel se encontraron estas inconsistencias, que conviene corregir en la próxima versión del Entregable 3:

1. **Product Backlog (3.2):** la columna Sprint no coincide con el cronograma. Debe quedar: HU-01 a HU-03 → 4; HU-07 y HU-08 → 5; HU-09 a HU-12 → 6; HU-04 a HU-06 → 7; HU-14 → 8; HU-13 → 9; HU-15 → 3 y 4.
2. **Sección 4:** dice “nueve sprints” pero son diez.
3. **Regla de redondeo:** el Entregable 1 dice redondear y RF-18 dice truncar.
4. **Canales de aviso:** el Entregable 1 menciona correo y tareas no entregadas; los siguientes solo WhatsApp por promedio y faltas. Aclarar qué quedó.
5. **Entregable 1:** dice “cuatro roles” y enumera cinco.
6. **Wiegers:** RF-29 aparece en los grupos 6 y 13, y RF-35 en los grupos 13 y 15.

7. **Texto de Wiegers:** dice que la planificación se construye antes que los reportes, pero en el cronograma va después.
8. **Justificación de faltas:** definir si la registra Inspección o el estudiante (RF-21).
9. **Año lectivo:** el prototipo usa fechas del régimen Costa (mayo a marzo); confirmar el régimen del colegio y unificar con el supuesto de “2025-2026”.
10. **Nueva sección:** agregar al Entregable 3 la arquitectura de microservicios, las tecnologías por servicio y la estrategia de pruebas de este documento.

Glosario

API Gateway
Punto único de entrada que dirige cada petición al servicio correcto y aplica seguridad común.

CI/CD
Integración continua (compilar y probar cada cambio) y entrega/despliegue continuo (publicar automáticamente lo que pasó las pruebas).

DLQ
Dead Letter Queue: cola donde quedan los mensajes que no se pudieron procesar.

JWT / JWKS
Token firmado que identifica al usuario / lista de claves públicas para verificarlo.

Outbox
Tabla donde se guardan los eventos en la misma transacción que el dato, para publicarlos después sin perderlos.

p95 Tiempo por debajo del cual está el 95 % de las respuestas.

PWA
Aplicación web que se instala y funciona sin conexión.

SBOM
Lista de componentes de software que contiene una imagen.

SAST / DAST
Análisis de seguridad del código fuente / de la aplicación en ejecución.

Testcontainers
Librería que levanta contenedores reales (PostgreSQL, RabbitMQ) durante las pruebas.